The Asset and the Target

Concentration, Consequence, and the Generation Mix That Serves Continuous Load

Parts One through Eight — complete draft

Third companion to the Generation Mix and Always-On Economy papers

Version 0.15 — DRAFT

17 October 2026

Tim Roxey

Eclectic Technologies

T. Roxey, "The Asset and the Target: Concentration, Consequence, and the Generation Mix That Serves Continuous Load," Version 0.15, Eclectic Technologies, October 2026.

Concept DOI: https://doi.org/10.5281/zenodo.22818331

Version DOI: https://doi.org/10.5281/zenodo.23439583

Current version: https://scubanuke.github.io/publications/papers/The_Asset_and_the_Target.pdf

Complete draft.

Primary audience: the investment community addressed by the two companion documents; the policy and infrastructure-protection treatment is Keeping the Lights On for Everything That Can't Go Dark.

Methodology is published and site-specific detail withheld. No utility, facility or site is named.

Erratum to Versions 0.12 and 0.13

Versions 0.12 and 0.13 of this paper, deposited under version DOIs 10.5281/zenodo.22818332 and 10.5281/zenodo.23017306 (concept DOI 10.5281/zenodo.22818331), said in Part Six, under "The islanding boundary," that "The Predictive UFLS Ride-Through position paper states the design consequence in its Section 4.6." That pointer was in error on its date. The section it named claimed local sovereignty for a ride-through extension that decides on a learned Gaussian process posterior, which does not meet the Foundational Definition FD-LS, and that paper withdraws the claim at its version 0.8. The same passage stated FD-LS's three conditions in the form then issued: "it acts on physical law, it is colocated with what it protects, and no authority that can reach it from outside sits in its decision path." That statement was accurate on its date and is superseded rather than in error. The October 2026 Edition of FD-LS states the physical-law condition as a fixed function of measured quantities and setpoints, with no learned element, and the third condition in two limbs. The passage now states the conditions as issued and draws the design consequence itself. Nothing else in the paper changes, and the earlier versions stand as deposited.

A Note on Scope

This document develops an analytical method and applies it to a generic, illustrative portfolio. It names no utility, no facility, no interconnection, and no site. The governing test, applied throughout and applied again to every later part, is whether a passage would materially assist an attack on a specific facility. Methodology, architecture, and stated gaps are published. Site-specific detail is not.

That distinction is not a courtesy. The argument below turns on the fact that a targeting ranking derived from public physics and public economics is already available to anyone who cares to derive it. What is withheld is never the method — it is the particulars that convert a method into an operation.

This paper is divided into Parts and numbers none of its own sections. Every section number in it belongs to another document: "Section" refers to the Generation Mix technical paper and "§" to DBA-MA-DiD, unless another document is named with it.

Part One: The Identity

What the framework established

Two companion documents precede this one. *Generation Mix Optimization Under Competing Objectives* develops a parametric framework evaluating generation sources across three objectives that the hyperscale data center load profile forces to be considered simultaneously: carbon intensity, capital recovery, and physical resilience. *Powering the Always-On Economy* presents the same argument without the derivations.

Their shared conclusion is narrow and well supported. For continuous, large-scale, low-carbon, resilient demand, nuclear generation satisfies every objective within acceptable ranges, and no other source does. The finding rests on parameter values applied consistently rather than on any assumption favorable to nuclear power. Section 5.4 of the technical paper states the strongest form of it: nuclear is Pareto-dominant, there is no allocation of nuclear that worsens either objective, and the optimization therefore pushes nuclear allocation to its maximum feasible level, bounded only by permitting, siting, capital, and grid integration limits.

That is the result. This document is about what follows from it.

The dual was already specified

Section 7.3 of the technical paper identifies two extensions as forward work. The second is the adversarial dual, and it is specified there with more precision than a forward-work note usually carries. The primal optimization asks which allocation minimizes carbon intensity subject to

resilience and demand constraints. The dual asks the mirror question: given a portfolio, which unit should an adversary destroy to maximize grid consequence? The paper names the objective. The highest-consequence target is the unit whose loss simultaneously maximizes the generation-load imbalance, minimizes the remaining system inertia, and imposes the longest recovery timeline on the affected capacity.

Throughout this document the word *dual* names a contest between two parties reading the same facts. The primal is the defender's problem: an owner, an investor or a planner choosing a portfolio to serve load at low carbon and high resilience. The dual is the adversary's problem: someone choosing which part of that portfolio to take away to cause the most harm. The two are posed over one parameter table, and nothing in either is hidden from the other side. The term is used in that plain sense and not in the formal sense of optimization theory, where a dual problem is a derived mathematical companion whose solution bounds the primal's. The claim here is narrower and more practical: the defender and the adversary are solving mirror-image problems with the same inputs, and the rest of this document follows from what happens when they do.

This document executes that forward work. It does not introduce the adversarial question; the framework raised it and deferred it.

Running the dual

The dual reads three quantities. It wants the largest single contribution to the imbalance following a loss, which is a function of unit size and of how reliably that unit was producing. It wants the largest reduction in remaining system inertia, which is a function of the inertia constant of the machine removed. And it wants the longest interval before the lost capacity returns, which is a function of the recovery timeline for the components whose destruction takes the unit offline.

Now consider which source the primal optimization selected, and why.

It selected nuclear because nuclear carries the highest inertia constant of any source in the analysis, at H equal to 6.0 seconds. It selected nuclear because nuclear carries the longest fuel independence of any dispatchable source, at 270 days against 0.5 days for a combined-cycle gas turbine. It selected nuclear because nuclear's productive utilization, the product of capacity factor and availability factor, stands at 0.83 and places it at the top of the utilization range, which is what drives the capital recovery result. Each of these is a reason the primal wants more of it.

Each is also a reason the dual wants to take it.

The unit with the highest inertia constant is the unit whose removal most reduces remaining system inertia. The unit with the highest productive utilization is the unit most likely to be producing at full output at the moment of attack, which maximizes the imbalance its loss creates. Fuel independence is the one parameter that cuts the other way at first glance — it makes the unit harder to neutralize through supply denial — but the dual is not required to choose denial. Against an adversary willing to act physically, high fuel independence means only that the

physical route is the route that works, and the recovery timeline of eighteen to twenty-four months then governs how long the consequence persists.

The identity, stated plainly

The primal and the dual are not two analyses that happen to point at the same asset. They are two readings of the same parameter table, running in the same direction.

The properties that make a generation unit valuable to the grid — high inertia, high productive utilization, long fuel independence, and the long recovery timeline that is the price of the heavy plant delivering the first three — are, without remainder, the properties that make it the highest-consequence target. There is no separating them, because they are not two sets of properties. They are one set, evaluated by two parties with opposite intentions.

This is why the finding cannot be dismissed as irony or as a rhetorical trick. It is derivable. Given the parameter table and the objective function stated in Section 7.3 of the technical paper, the dual selects the unit the primal recommends, and it selects it for the reasons the primal recommends it.

The optimization that maximizes the portfolio's value is, in the same act, maximizing the consequence of a single successful attack on it. The framework has no term in which that fact appears.

The method that does have a term for it

That sentence raises an obvious question, and the answer is not that the problem is unaddressed. It is that the method addressing it sits outside the framework the two companion papers use.

Nuclear engineering has roughly seventy years of practice in consequence-based design. The design basis states what a facility must withstand, the accident analysis establishes what happens when it is pushed past normal operation, and the credited barriers are the things the safety case relies on to keep consequence bounded. The discipline is mature and it works, and everything it relies on was built around events that were not chosen by anyone.

The extension this analysis needs asks a single question of that body of work: what changes when the initiating event is selected rather than random? Four properties of deliberate action each break a specific assumption the accident analysis depends on.

Intentionality removes the premise that an accident does not choose. A random failure takes whichever component fails; an adversary takes the component whose loss costs most, which means the barrier the safety case leans on hardest is the barrier most likely to be attacked first. Persistence removes the sequence of event followed by aftermath. Accident analysis assumes recovery begins once the event ends; against an adversary who is still present, recovery is conducted under continuing action. Precision breaks the indiscriminacy that defense in depth quietly exploits — layered barriers work well against a threat that cannot aim, and less well

against one that can. And denial removes the assumed availability of everything external: fuel deliveries, spare components, mutual aid, offsite power, the arrival of people.

What this produces is not a different answer about which asset to build. It is a statement of what that asset must withstand, and — this is the part that matters for an investment audience — an explicit residual wherever it cannot. A design basis extended this way does not promise that the asset survives. It says what it survives, what it does not, and which of those judgments are load-bearing for the contract the asset underpins.

The standing of this method should be stated honestly, because an investor who checks will find out anyway. It is pre-regulatory. It is not a standard, no one is obliged to follow it, and nothing in it has been reviewed by a regulator. Its claim on a reader is the quality of its reasoning and nothing else. What it does offer is that the analysis in the parts that follow applies an existing method rather than improvising one under the pressure of an uncomfortable finding. The instrument the parts that follow cite for the barrier-correlation method, DBA-MA-DiD, is an unpublished working draft. Version 0.2, issued on 22 September 2026, carries the constructs accepted from this paper: the count of independence classes and its reporting as a bounded range at §3.2, and the evaluation of consequence at the level of the class at §3.4. It has been issued as a working document and not published, so a reader who checks a citation will find that it resolves against the author’s own corpus rather than against a published object. That is a limitation of the citation and not of the reasoning behind it.

Part Two: Why the Obvious Escapes Fail

An identity of this kind invites two immediate responses. Both are wrong, and both are wrong in instructive ways.

The first escape: diversify away from the dominant asset

The first response is to conclude that concentration in the highest-value asset is imprudent, and to spread allocation across a wider set of sources so that no single loss is decisive.

This fails on the framework's own arithmetic. The sources available to absorb that allocation are dominated sources, and they are dominated because their parameter values are worse. Moving allocation to combined-cycle gas buys a carbon intensity of 490 grams per kilowatt-hour against nuclear's 12, and a fuel independence of half a day against 270 days — a portfolio that satisfies every current adequacy standard while remaining structurally unable to survive a supply interruption lasting longer than an afternoon. Moving allocation to wind or solar buys the lowest carbon intensity in the table and an inertia contribution of zero, which by the swing equation developed in Section 5.5 of the technical paper raises the rate of frequency decline following any generation loss, deepens the frequency nadir, and increases the probability that automatic under-

frequency load shedding operates before the decline is arrested. Load shedding, from the perspective of the continuous load, is an outage. It is not distinguishable from a supply failure by anyone downstream of the relay.

So the first escape does not reduce consequence. It redistributes it, raises cost, worsens carbon, and opens a second and entirely separate pathway by which the load loses power — one that does not require an adversary to attack the load's own generation at all, and that the load's operator cannot see coming.

There is a subtler error inside the first escape, and it runs two levels deep.

The first level is that the instinct to diversify conflates diversity of *source type* with granularity of *units*. The dual targets a unit, not a technology. A portfolio spread across four source types, each delivered by one large machine, leaves four large targets rather than one — an improvement, but a much smaller one than the language of diversification implies.

The second level is the one that matters, and it is where most of the intuitive appeal of modularity evaporates. Unit granularity itself conflates *count* with *independence*. Ten machines are ten targets only if defeating the second through tenth costs something like what defeating the first cost. Where the units share a design, a control architecture, a firmware image, a vendor update channel, a supply chain, an operating crew, a switchyard, or a heat sink, that marginal cost collapses toward zero. The work was all in the first one. Ten units under a single shared dependency are not ten targets. They are one target wearing ten bodies.

Hold both levels. Part Three shows that the framework does not distinguish any of them.

The second escape: decline to compute the ranking

The second response is to observe that a targeting priority ranking is dangerous information and to conclude that it should not be derived, or at least not written down.

This fails because the ranking is not proprietary. Every input to the dual is public. Inertia constants for machine classes are in the engineering literature. Capacity factors and availability factors are published by reliability organizations and regulators. Recovery timelines for large power transformers have been the subject of government assessments, industry reports, and academic analysis for more than a decade. The swing equation is physics and has been since the 1920s. Unit sizes and locations are in interconnection queues, environmental filings, and commercial mapping services that catalogue thousands of facilities together with ownership and interconnection detail.

An adversary in possession of the same public parameters and the same undergraduate power systems education derives the same ranking. Declining to write it down withholds nothing from them.

What it does withhold is the ranking from the defender, who needs it to allocate protection that is always finite. The asymmetry runs entirely the wrong way: the party that must choose where to

spend a hardening budget is the party disadvantaged by refusing to do the analysis, and the party choosing where to strike is unaffected. This is the same reasoning that governs publishing methodology while withholding site-specific detail. The method is not the secret. The particulars are.

What the escapes have in common

Both escapes treat the target designation as an argument against holding the asset. Neither examines whether the asset can be held differently.

That is the move Part Three makes available, and the reason it becomes available is that the framework, as written, cannot represent the difference between holding an asset well and holding it badly.

Part Three: What the Framework Cannot See

The resilience function is where the properties discussed in Part Two would have to appear if they appeared anywhere. Section 5.2 of the technical paper defines it as a vector of three components rather than a scalar composite, on the stated grounds that weighting inertia deficiency against fuel denial against component destruction requires value judgments outside the paper's scope. That reservation is sound. But the vector has a different and unstated limitation, and it is the one that governs everything here.

The limitation has a single name. **The resilience function cannot express shared dependency.** It evaluates the properties of sources, and dependency is not a property of a source. It is a property of how sources are assembled, connected, supplied, controlled, and crewed. Two portfolios built from identical machines can have completely different dependency structures and the function will score them identically, because there is no term in which the difference can be written.

That limitation shows up in two places. They look like separate problems and they are the same problem.

Unit independence

Take the three components as written.

System inertia is the capacity-weighted average inertia constant across the portfolio, weighted further by capacity factor to reflect that a machine contributes inertia only while synchronized. Portfolio fuel independence is the demand-weighted minimum across sources. Portfolio recovery timeline is the capacity-weighted average across transmission-connected sources.

Now construct three portfolios serving an identical load.

The first uses a single large nuclear unit. The second uses ten nuclear units of one-tenth the capacity, of one design, on one site, sharing a control building, a control architecture, a firmware baseline, a vendor maintenance channel, a switchyard, an ultimate heat sink, and an operating crew. The third uses ten units of the same capacity, deliberately separated: diverse control implementations, independent heat removal, separated electrical connections, staggered procurement across suppliers, and crews that do not all follow the same procedure at the same moment.

Every term in the resilience function is identical across all three. The inertia constant is a property of the machine class, so the weighted average does not move. The fuel independence is a property of the fuel cycle, so the minimum does not move. The recovery timeline is assigned by source type in the parameter table, so the weighted average does not move. The framework returns one resilience vector and says these are the same portfolio.

The dual separates them decisively, and not in the way a count of machines would suggest.

Against the first, one successful attack removes the whole contribution — the entire imbalance, the entire inertia, the whole of that capacity for the eighteen to twenty-four months the recovery requires.

Against the second, it depends entirely on what the attack is. A physical attack on one module removes a tenth, and an adversary wanting the rest must repeat the act nine times, paying nine times in cost, time, and exposure. But an attack that reaches the shared dependency — a firmware image, a vendor update channel, a control architecture, the switchyard, the heat sink — removes all ten at once, and the marginal cost of modules two through ten is approximately nothing. The second portfolio is ten targets against one class of attack and one target against another. It is worse than the first portfolio in one specific respect: ten instances of a single design offer ten opportunities to discover the flaw and ten places to rehearse using it.

Against the third, the shared-dependency route has been deliberately narrowed, and the adversary is closer to facing ten genuinely separate problems.

What the dual is actually sensitive to, then, is not the number of units. It is **the number of units that must be independently defeated** — which is to say the number of independence classes, not the number of machines. Call that the portfolio's effective granularity. One large unit has an effective granularity of one. Ten modules sharing their controls and their site have an effective granularity close to one; they are merely a more expensive way of owning a single point of failure. Ten modules with independence actually purchased approach ten, and the purchase is neither automatic nor free.

This is the correction that matters most for anything downstream. Modularity does not deliver granularity. Modularity creates the opportunity to buy granularity, and whether a given developer bought it is a question with a concrete answer, asset by asset. A campus of twelve identical modules on one shared instrumentation and control architecture has bought the appearance of distribution and none of the substance.

There is a further detail that sharpens the point rather than softening it. In reality the second portfolio probably does not even share the first's recovery timeline, and not in its favor: shared transformer spares, shared crews, and a common design mean that a common-mode event damages ten units with one pool of replacement components and one set of people to install them. The framework cannot express that either, because the recovery parameter is assigned per source type rather than per configuration.

And there is one place — exactly one — where the parent framework already reasons correctly about shared dependency. Portfolio fuel independence uses a **minimum operator, and Section 5.2 of the technical paper justifies** it on the grounds that a portfolio's independence is constrained by its least independent component. That is precisely the logic this section is asking for, applied to fuel and applied nowhere else. Inertia and recovery use capacity-weighted averages, because the framework does not conceive of them as shared. On a campus they are shared: common firmware determines whether the inertia is delivered, common spares and crews determine whether the recovery happens. The reasoning the other two parameters need is already sitting inside the third.

Control independence

The second manifestation is already half-identified in the parent document, which makes it easier to state and harder to dismiss.

Section 5.6 of the technical paper establishes the distinction cleanly. Physical inertia from a synchronous machine is a property of rotating mass. It cannot be disabled by a firmware update, withdrawn by a remote command, or manipulated through a communication interface, because it operates below the control system layer entirely. Synthetic inertia from a grid-forming inverter has none of those protections: it is a control algorithm executing on a digital signal processor, dependent on proprietary firmware, reachable in principle through whatever interfaces the manufacturer implemented. The section documents the supply chain concentration that makes this concrete, and observes that an inverter withdrawing its inertia response at a moment of stress would be indistinguishable from normal behavior until the moment of consequence.

The analysis is correct. The difficulty is what the framework does with it. Section 5.6 of the technical paper resolves the problem by treating inverter-based inertia as zero, and argues that the H = 0 treatment in the parameter table correctly reflects effective inertia contribution in a supply-chain-aware assessment.

For the grid as it stands, that is right, and it errs in the correct direction. But it is a workaround rather than a term. It holds only while synthetic inertia is negligible in practice. As grid-forming inverters become standard rather than exceptional — and the direction of deployment is not in doubt — H = 0 stops being conservative and becomes simply inaccurate, and the framework will need to distinguish inertia that is physically guaranteed from inertia that is contingent on a device behaving as specified. The resilience vector has no dimension in which that can be recorded.

Read alongside the previous section, this is not a second problem. It is the first problem in its purest form. A firmware baseline common to every unit in a fleet is a shared dependency with no physical separation at all — no distance, no wall, no separate building, nothing that a determined adversary must cross twice. Diverse hardware with identical software has an effective granularity of one, and the resilience function will score it as a well-distributed portfolio. The commandability question and the independence question are the same question asked of different layers.

What follows

The framework recommends concentration in the highest-value asset and has no term in which shared dependency — the thing that decides whether distribution is real or cosmetic — can be expressed. That is not a defect of reasoning. The paper is explicit that its resilience treatment is a vector rather than a composite and that certain judgments were deliberately excluded. It is a gap in expressive capacity, and it is closable.

The remainder of this document closes it. Part Four develops the adversarial dual against a generic illustrative portfolio, so that the ranking the framework implies can be seen rather than inferred. Part Five proposes the missing term — effective granularity, defined over independence classes rather than machine counts, and carrying control independence as one of the dependencies it counts — and shows what the resilience vector looks like once it is present. Part Six applies the extended framework to the architecture it selects, which is not a retreat from nuclear concentration but a conditional endorsement of a particular way of distributing it: modular generation behind an islanding boundary, with the independence actually purchased rather than assumed. Part Seven states what adopting this requires of a developer or investor, in sequence and at cost. Part Eight states what remains unresolved and who can resolve it.

The conclusion of the two parent documents is not overturned by any of this. Nuclear remains the only source satisfying every objective the framework can express. What changes is the qualifier, and the qualifier is load-bearing: it passes every test the framework can express, and shared dependency is a test the framework cannot express. Building to the recommendation without carrying that qualifier is how a portfolio arrives at maximum measured resilience and maximum unmeasured exposure in the same moment — and how a developer pays for twelve reactors while owning the risk profile of one.

Part Four: The Dual, Worked

The portfolio

What follows is a constructed portfolio. Its capacities are round numbers chosen for arithmetic legibility, its composition is invented, and it corresponds to no interconnection, utility, or region.

Nothing in this section should be read as a description of an existing system. The purpose is to show what the calculation does, using the parameter values published in Table 1 of the technical paper.

Call it System A. It has 6,000 megawatts of nameplate capacity serving a mixed regional load that includes a block of continuous data center demand. One nuclear unit of 1,200 megawatts. Four combined-cycle gas units of 400 megawatts each. One large hydroelectric station of 600 megawatts. Twelve hundred megawatts of wind and fourteen hundred of solar.

Applying the system inertia formula of the technical paper's Section 5.2 — the capacity-weighted average inertia constant, weighted further by capacity factor to reflect that a machine contributes inertia only while synchronized — System A carries a baseline H_system of 3.86 seconds.

The dual at unit level

The dual asks which single unit an adversary should remove. It reads three quantities: the output lost, which drives the imbalance; the inertia removed, which governs how violently frequency responds to that imbalance; and the recovery timeline, which governs how long the loss persists. Expected output at the moment of attack is taken as nameplate multiplied by the product of capacity factor and availability factor, since a unit that is not running cannot be lost.

Candidate target	Output lost, MW	Inertia lost, MW-s	H_system after, s
Nuclear unit, 1,200 MW	996	5,976	2.64
Wind fleet, 1,200 MW	408	0	4.48
Solar fleet, 1,400 MW	350	0	4.37
Large hydro, 600 MW	222	888	3.85
One CCGT unit, 400 MW	192	864	3.81

The nuclear unit wins by a distance, and it wins on every term at once. It sheds the largest block of output because its utilization is the highest in the table at 0.83. It removes by far the most stored rotational energy because its inertia constant is the highest at 6.0 seconds. And its recovery runs to eighteen to twenty-four months, bounded by a custom high-voltage transformer with no strategic reserve behind it.

The compounding is the part worth dwelling on. System inertia falls from 3.86 seconds to 2.64, a reduction of nearly a third, *at the same instant* a thousand-megawatt imbalance appears. The swing equation of the technical paper's Section 5.5 makes the rate of frequency decline proportional to the imbalance and inversely proportional to system inertia. The dual's preferred target is the one unit whose loss pushes both variables the wrong way simultaneously. This is not

three separate advantages that happen to coincide. It is one machine whose physical properties make it the load-bearing element of the system's frequency response.

Two entries in the table repay a second look.

The wind and solar fleets *raise* remaining system inertia when removed — 3.86 goes up to 4.48 and 4.37 — because taking zero-inertia capacity out of a weighted average lifts the average. An adversary attacking wind or solar makes the surviving grid more robust to the next disturbance, not less. Those fleets are imbalance-only targets, with recovery timelines of three to twelve months rather than eighteen to twenty-four. They are genuinely less attractive, and the framework says so correctly.

The hydro station ranks fourth on output and near-last on inertia, but it carries the longest recovery timeline in Table 1 at twenty-four to thirty-six months. It is a persistence target rather than a magnitude one. A ranking that collapsed the three terms into a single score would hide that distinction; keeping the resilience function a vector preserves it, which is a point in favor of the parent paper's choice.

Now partition the portfolio

Replace the single nuclear unit with twelve modules of one hundred megawatts, on one site, of one design, sharing a control building, a control architecture, a firmware baseline, a vendor maintenance channel, a switchyard, an ultimate heat sink, and an operating crew. Call this System B. Nothing else changes.

Run the unit-level dual again and the answer transforms. The best single target is now one module: 83 megawatts of output and 498 megawatt-seconds of inertia. The worst single loss has fallen by roughly ninety-two percent. On the face of it, System B is a dramatically hardened portfolio.

Now run the dual over independence classes instead of units. System B's twelve modules constitute one class, because any of the shared elements listed above defeats all twelve. The class-level answer is 996 megawatts, 5,976 megawatt-seconds, and a fall in system inertia to 2.64 seconds — identical, to the digit, to System A.

The quantity that improved by ninety-two percent is the one that does not describe the exposure. The quantity that describes the exposure did not move at all. System B bought twelve reactors and retained the risk profile of one.

The cheaper second place

The four gas units look, at unit level, like four separate 192-megawatt targets sitting well down the ranking. Suppose they share a gas lateral, which is an ordinary arrangement rather than an exotic one. They are then a single independence class, and the class-level figures are 768

megawatts of output and 3,456 megawatt-seconds of inertia — second in the ranking and within striking distance of first.

The parameter that makes this bite is fuel independence. Table 1 gives combined-cycle gas an FID of half a day. The class does not fail at the moment the lateral is cut; it fails about twelve hours later, when linepack runs out. An adversary who wanted the consequence without the difficulty of reaching a hardened reactor building has a second option that is cheaper to execute, demands different skills, and carries a different attribution profile — and the unit-level ranking conceals it entirely, because it never assembles the four machines into the thing that actually fails together.

For an investor, this is the practical yield of the whole exercise. A hardening budget allocated against a unit-level ranking would spend everything on the reactor and nothing on a pipeline branch that delivers most of the same consequence for a fraction of the adversary's cost.

What independence actually buys

Now take System C: the same twelve modules, but deliberately separated into three groups of four, with distinct control implementations, independent heat removal, separate electrical connections, procurement staggered across suppliers, and crews that do not all work to one procedure at one moment.

The worst class is now four modules: 332 megawatts, 1,992 megawatt-seconds, and system inertia falling from 3.86 to 3.57 rather than to 2.64. Against the best available coordinated attack, System C's exposure is roughly a third of System A's or System B's. That is a real reduction, purchased with real money and real engineering, and it is the only one of the three configurations where the money bought what it appeared to buy.

What the three systems show

Under the resilience function as written, Systems A, B and C return the same vector. Same weighted-average inertia constant, same portfolio fuel independence, same weighted-average recovery timeline. The framework cannot distinguish them.

Under the dual computed over independence classes, A and B are indistinguishable from each other and C is three times better than either. The dual cannot see anything else.

Two analyses of the same three portfolios, one of which is blind to the distinction that matters and the other of which consists of nothing but that distinction. That is the gap Part Five closes.

Part Five: The Missing Term

What has to be added is not a number

The natural instinct is to add a count of units to the resilience function. Part Four shows why that fails: System B has twelve units and the exposure of one. A count measures the thing that is easy to observe rather than the thing that governs the outcome.

What the resilience function needs is not a scalar at all. It needs a **partition** — a statement of which units fall together, from which the decision-relevant quantities follow.

Declared dependencies and independence classes

The construct begins with a declared set of shared dependencies, and that declaration is not this paper's invention. The design-basis method carries it as the **barrier-independence ledger** of DBA-MA-DiD §3.2, which requires every credited barrier to be mapped to the substrate its safety function depends on, sorted into four dependency classes — physical, electrical, digital and organizational — with a record, for each substrate element, of whether it is shared across barriers or across units. That last field is the whole input to what follows. The obligation to produce it is the method's, not this paper's.

What this paper supplies is the generation-sector instantiation. For a generation portfolio the candidates are unsurprising, and they sort into the ledger's four classes readily: physical site and ultimate heat sink; electrical connection and switchyard; control system architecture, firmware baseline, and vendor maintenance and update channel; and operating crew and procedure, fuel supply path, and the pool of long-lead spares and the people who can install them.

Two units are **dependent** when they share at least one declared dependency whose compromise defeats both. Dependency defined this way is reflexive and symmetric but not in general transitive, so for partitioning purposes take its transitive closure: if A and B share a switchyard and B and C share a firmware baseline, all three fall into one class. The closure groups more aggressively than strict logic requires, and that conservatism is deliberate — a portfolio should have to earn a claim of independence rather than receive it by default.

The closure and the partition it produces are a general result rather than a generation-sector one, and this paper does not claim them. DBA-MA-DiD v0.1 did not carry them: its second move enumerates minimal substrate-defeat sets, which is a different object computed from the same ledger, and its output is a set of governing sets and a test applied to each rather than a count. The construct was accordingly raised with the owner of that instrument on 14 September 2026, accepted, and carried into version 0.2 on 22 September 2026, where §3.2 takes the transitive closure of the ledger's shared-substrate relation and reports the count of independence classes as the ledger's output. What this paper develops is only what is particular to a generation portfolio.

The result is a partition of the portfolio into **independence classes**. Define the portfolio's **effective granularity**, G_eff, as the number of classes. System A has a nuclear G_eff of one. So does System B. System C has three.

The adversarial resilience vector

With the partition in hand, the dual becomes computable, and it computes over classes rather than units. For each class, take the aggregate output at risk, the aggregate inertia contribution, and the class recovery timeline. Then define

R_adv(x) = (ΔP_max, H_remaining_min, RT_max, G_eff)

where ΔP_max is the largest aggregate output attributable to any single class, H_remaining_min is the lowest system inertia surviving the loss of any single class, RT_max is the longest class recovery timeline, and G_eff is the class count.

This is the adversarial dual of the technical paper's Section 7.3, made concrete. The parent paper specified the objective — the unit whose loss simultaneously maximizes the imbalance, minimizes remaining inertia, and imposes the longest recovery — and the only thing added here is the observation that the objective should range over classes rather than machines, because a class is what actually fails together.

Reported alongside each other, the two vectors answer different questions. R(x) describes what the portfolio is made of. R_adv(x) describes what its worst coordinated loss costs. A portfolio can score well on the first and badly on the second, and System B is exactly that portfolio.

What a low effective granularity obliges

A number that produces no obligation is a number nobody acts on, and this one has an obligation attached. It is not this paper's to invent. The design-basis method states it at DBA-MA-DiD §3.4, which holds that adequacy under deliberate action is not restored by adding redundant barriers — an adversary who selects correlations simply re-selects to include the added one — but by ensuring that every governing set contains at least one element **independently protected** against defeat at the relevant threat tier, where independent means the protecting measure is not itself defeated by the same action set that defines the set. Where no such element exists, §3.5 requires the consequence to be enumerated and carried forward as a declared residual rather than assumed away.

Read onto a portfolio that is a short and testable instruction. Where G_eff is low because a dependency is shared, the owner either protects one element of that dependency in a way the same attack does not reach, or writes the exposure down as a residual and carries it. **Adding further units that share the dependency is not one of the options** — which is the same result Part Four reached from the arithmetic, and is worth having twice, because the instinct to answer a concentration finding with more units is strong and the method forecloses it in terms. Buying separation is a different act from adding units, and it is the act Part Six is about.

The residual limb is the one an investor should care about most, because it is the one that is checkable from outside. A developer who can produce a declared dependency set, a G_eff computed from it, and a written residual for every class carrying no independently protected element has done the analysis. A developer who produces the first two and no residual has not finished it, and the absence is visible without any access to the site.

The minimum-operator principle, generalized

Part Three observed that the parent framework already reasons this way in one place: portfolio fuel independence takes a minimum, justified on the grounds that a portfolio is constrained by its least independent component. That instinct is correct and it is under-applied.

The general principle is that any component describing a **shared** vulnerability must take a worst case over classes, not a capacity-weighted average over units. An average answers the question "what is this portfolio typically like." An adversary does not sample the portfolio at random. Averaging is the right operator for a question about expected behavior and the wrong operator for a question about a chosen event, and inertia and recovery are currently averaged for no reason other than that the framework was not asking an adversarial question when they were specified.

So alongside H_system and RT_HVLLC_portfolio, a portfolio should report H_remaining_min and RT_max. The first pair says what the portfolio ordinarily provides. The second says what survives the worst single coordinated loss.

Control independence needs no separate term

Part Three treated commandability as a second manifestation of the same limitation. The partition construct vindicates that reading, because a firmware baseline is simply one element of the declared dependency set, and a common baseline groups every unit running it into one class regardless of how far apart the machines stand.

This yields a result worth stating plainly: **diverse hardware running identical software has an effective granularity of one.** Physical separation that is not matched by control separation buys distance and nothing else. It is also the cheapest independence to lose and the hardest to notice losing, because a fleet standardizes its firmware for excellent operational reasons — maintainability, training, spares, regulatory consistency — and every one of those reasons is sound right up until the moment it is the mechanism of a common-mode failure.

No fourth dimension is required for it. It is a dependency, it belongs in D, and it partitions like any other.

What this does to the parent framework's results

Nothing in this disturbs the directional findings of the technical paper. Nuclear's inertia constant remains the highest in Table 1, its fuel independence the longest among dispatchable sources, its productive utilization the top of the range, and its carbon intensity second lowest. On every component of the original resilience vector and on the climate objective, the Pareto dominance result of Section 5.4 stands untouched.

What changes is that a fourth axis becomes visible, and on that axis a concentrated portfolio scores badly while a properly separated one scores well — at the same nuclear allocation. The recommendation therefore refines rather than reverses. It does not become "hold less nuclear." It becomes **maximize nuclear allocation, and buy effective granularity while doing it**, on the understanding that the second half is a separate purchase that modular construction makes possible and does not deliver.

That is a more useful instruction than the one it replaces, because it is actionable by the party receiving it and it distinguishes between two portfolios that the original framework would have priced identically.

The limits of the construct, stated first rather than last

The partition is only as good as the declared dependency set. An undeclared dependency does not reduce G_eff — it is simply invisible, and the portfolio scores as though the independence were real. This is the construct's central weakness, and there is no analytical remedy for it. The remedy is procedural: the declaration must be written down and reviewable by someone other than its author, because a developer who declares a thin dependency set receives a flattering number, and a flattering number obtained that way is worse than no number at all. That remedy is general rather than particular to portfolios. It was raised with the design-basis method’s owner on 14 September 2026 and is now stated at DBA-MA-DiD §3.2, which requires the ledger to be written and reviewed by someone other than its author. It is repeated here because the construct cannot be used safely without it rather than because this paper owns it.

The transitive closure is conservative and will sometimes group units that a careful analysis would separate. That is the intended direction of error.

G_eff is a screening construct, not a probability. It says nothing about how likely any attack is, and it should never be presented as though it did. It answers a bounded question — if one coordinated loss occurs, how much can it take — and that is the question a design basis asks.

It should also be said plainly that the declaration will be built largely from estimates. Some entries are close to fact: whether two groups of units share a switchyard, a heat sink, or a substation can usually be read from drawings. Others are judgments: whether two control implementations are genuinely independent, whether an update path truly stops at one group, how far a shared spare pool stretches a recovery. Those depend on vendor information the

developer may never fully see and on a view of what a capable adversary can reach. A real portfolio will therefore not yield a single, specific value of G_eff. It will yield a bounded range — three classes if the two control implementations are independent, one if they are not — together with the judgments on which the answer turns. Reported that way, the construct does not lose its usefulness. It gains it, because the reader is shown exactly which assumption the resilience claim rests on, and that is the assumption a reviewer should test first. The design-basis method has since adopted this form. DBA-MA-DiD §3.2 requires the count to be reported as a bounded range, the upper bound taken with every judged independence holding and the lower with every one failing, and permits a single figure only where review has shown that no judgment moves the result.

And the declaration itself is the point at which this analysis meets the method described at the end of Part One. Declaring which dependencies are shared is the same discipline as declaring which barriers a safety case credits: an explicit statement, made in advance, of what the argument rests on, so that a reader can test it rather than infer it. A portfolio whose owner cannot produce that declaration has not been shown to be fragile. It has been shown to be unexamined, which for a twenty-year commitment is not a materially better position.

Part Six: The Architecture the Extended Framework Selects

What the recommendation now says

Part Five ended on a refined instruction: maximize nuclear allocation, and buy effective granularity while doing it. This part applies the extended framework to the architecture that instruction points toward. It begins by stating the endorsement exactly, because its value to an investor lies in its conditions.

The endorsement is for modular nuclear generation serving continuous load from behind an islanding boundary, with independence between groups of modules actually purchased rather than asserted. It is conditional in a specific sense. Each condition is a claim about the declared dependency set, and each can be tested with the partition Part Five defined. The claim holds if the transitive closure over the declared set returns the class count the developer says it bought. It fails if it does not. Nothing in the endorsement rests on the developer’s description of the architecture. It rests on the ledger.

That matters because the architecture introduces an element Systems A, B and C did not have. An islanding boundary is a new piece of shared equipment, placed deliberately at the point where every module meets the grid. It is the reason the architecture is attractive. As the arithmetic below shows, it is also the cheapest way to lose everything the separation bought.

The islanding boundary

The islanding boundary is the perimeter within which the campus — its generation, its load, and the equipment that joins them — can keep operating when its connection to the wider grid is lost or deliberately opened. At the boundary sit the point of common coupling, the breakers and protection that open it, and an island controller that manages the transition and holds the island's frequency and voltage once it has separated. Inside, the nuclear modules act as the island's frequency anchor. The synchronous inertia that Part Four showed to be the load-bearing element of a regional system's frequency response is, on the island, very nearly the whole of it.

Defining the boundary is mostly a matter of stating what crosses it, because every crossing is a dependency the ledger must record. Fuel does not cross it for nuclear, and that property is what makes the architecture coherent: with 270 days of fuel independence, the island is not waiting on a delivery. Control authority crosses it wherever any function inside can be commanded, permitted, updated or configured from outside. Spares cross it, because the long-lead components that return a damaged module to service are held somewhere, and usually not inside. Crews cross it, since the people who operate, maintain and restore the island arrive from outside. The ultimate heat sink may or may not cross it, depending on siting. Communications cross it too, both the operational links and the vendor channels that carry maintenance and updates. No boundary exists across which only electrical power flows. The question is which crossings have been declared.

The boundary also has to be analyzed in both directions, because it can fail in both.

In the first direction, a grid disturbance arrives at the boundary from outside, and the record here is recent. NERC's incident review of 8 January 2025, *Considering Simultaneous Voltage-Sensitive Load Reductions*, examined an event of 10 July 2024 in an area of the Eastern Interconnection with a high concentration of data centers. A lightning arrester failure on a 230-kilovolt line produced six faults in eighty-two seconds. About 1,500 megawatts of load transferred to backup power, and about 1,260 megawatts stayed off for hours. Frequency peaked at 60.047 hertz and voltage at 1.07 per unit. The load behaved as it was designed to behave. Because the protection at each facility was set to respond to the same disturbance in the same way, the facilities responded together. That is the grid-to-island failure in its most instructive form: the load did what it was designed to do and still dropped as a block.

Two years later the same response came at more than twice the scale. On 22 July 2026 a single conductor pulled free of its compression sleeve on a 230-kilovolt line in the same interconnection, and data center load transferred to backup power in two waves sixty-seven seconds apart. The regional transmission organization put the load transferred at about 3,869 megawatts, and frequency peaked at 60.092 hertz. The Eastern Interconnection's resource loss protection criterion for 2026 is 3,875 megawatts. A routine equipment failure removed almost exactly the contingency the interconnection is sized to withstand, and it did so as a family of facilities responding together, not as a single unit.

In the second direction, the island is the source of the disturbance. A campus that trips removes its net interchange from the grid in an instant, whether its boundary opened under attack, its island controller misoperated, or its load transferred to backup. Where the campus exports, it removes its synchronous inertia in the same instant. The grid then experiences the campus as a single large contingency whose size is set by decisions made inside the fence.

The protective functions at the boundary are therefore where local sovereignty earns its place. FD-LS Section 3.2 sets three conditions, which must hold together. The function's decision is physics: a fixed function of what it measures and its setpoints, with no learned element. It is sited with what it protects. And no reachable authority sits in its path. At the moment of acting, nothing that can be reached from outside can withhold, delay or induce its action, whether by command or by corrupting what it measures. Ahead of acting, nothing reachable can alter its setpoints, logic or bypass state except through a governed act, which is a deliberate, authenticated and recorded act by a qualified human that no automated element can perform alone. The design consequence follows. A protective decision that depends on external power, external communications or operator action will fail under the very conditions it exists for. Applied here, the under-frequency, out-of-step and islanding-detection protection at the point of common coupling must decide and act on local measurements and local power, with no permissive awaited from a networked controller and with settings that change only through a governed act. A colocated relay that waits for a remote permissive is not locally sovereign, however local its hardware. Neither is a protective decision made by a learned model, however locally it runs. The Predictive UFLS Ride-Through position paper draws that second line in its Section 4.6, where the fixed-threshold floor can carry the property and the learned extension above it cannot.

One limit travels with that conclusion. FD-LS Section 4 states that local sovereignty does not decide where the Bright Line falls. A locally sovereign protective function does not exempt the actions taken above it from governance. The island controller's authority to dispatch, curtail, or reconfigure is a separate question from whether the boundary's protection is sovereign, and this paper answers only the second.

The boundary is itself a dependency

Here the architecture meets the construct Part Five built. The result is uncomfortable enough to be worth working through in numbers.

A single point of common coupling, or a single island controller, is a substrate element shared by every module behind it. The barrier-independence ledger of DBA-MA-DiD §3.2 records it as shared across units, and the partition joins every module it touches into one class. It does so however carefully the modules were separated from one another in every other respect.

Call the result System D. It is System C in every particular — twelve 100-megawatt modules in three groups of four, with distinct control implementations, independent heat removal, separate

electrical connections within the campus, staggered procurement, and crews that do not all work to one procedure at one moment — with one addition: a single islanding boundary through which all twelve connect to the grid. System E is the same campus with the boundary split into three points of common coupling and three island controllers, one for each group, with no protection, control or communications path shared between them.

The arithmetic uses Table 1 of the technical paper and the conventions of Part Four without change. System inertia is the capacity-weighted average inertia constant, weighted further by capacity factor. Output at risk is nameplate multiplied by the product of capacity factor and availability factor. The baseline for every system is the H_system of 3.86 seconds computed for System A. Every figure below was recomputed programmatically from those inputs, including the Part Four figures it repeats, and the recomputation agrees with Part Four to the digit.

System	Nuclear independence classes	Worst nuclear class, output lost, MW	Inertia lost, MW-s	H_system after, s
A: one 1,200 MW unit	1	996	5,976	2.64
B: twelve modules, all shared	1	996	5,976	2.64
C: three separated groups of four	3	332	1,992	3.57
D: System C behind one boundary	1	996	5,976	2.64
E: System C behind three aligned boundaries	3	332	1,992	3.57

System D returns System A’s figures to the digit, and transitive closure is what does it. The boundary puts module one in a class with module five and module nine, and the group structure then pulls in everything else. Everything System C bought in separated controls, heat removal, and procurement still holds against every attack but one. The one it does not survive is an attack on the single element the developer added to make islanding possible. An adversary no longer needs to reach any of the three control implementations. Against a boundary attack, the campus has an effective granularity of one.

The electrical detail sharpens this point rather than softening it. When the boundary opens and the campus load leaves with the island, the grid’s imbalance is the campus’s net export at that moment. That may be far smaller than 996 megawatts, and for a fully self-supplied campus it may be close to nothing. The inertia receives no such discount. All 5,976 megawatt-seconds leave the grid together, and the surviving system faces its next contingency at 2.64 seconds, whatever the interchange was. The load, meanwhile, depends entirely on an island held together by the same controller whose compromise defined the class. Where the campus exports, an

attack on the boundary is a magnitude attack on the grid. Where it does not, the attack still leaves the grid exposed at reduced inertia. In either case, it is a direct attack on the load.

System E recovers System C's figures, but only under two conditions the table cannot display. The first is that the boundaries align with the groups. Suppose the same campus were split into two boundaries of six modules each. Each boundary would straddle a group; closure would join the first boundary to the middle group and the middle group to the second boundary, and the campus would return to one class. It would have paid for two boundaries and received the protection of none. The second condition is that the three boundaries do not share what makes them boundaries. Three island controllers running one firmware image, configured from one engineering workstation or updated through one vendor channel are, in the ledger, one controller installed three times, and the closure again returns a class count of one. Splitting the boundary is necessary, but it is not sufficient.

The same arithmetic shows what an investor should expect once System E is achieved, and it is not rest. With the nuclear campus separated into three classes of 332 megawatts each, the nuclear class no longer leads the output ranking. If the four gas units share a lateral, as Part Four supposed, that class sheds 768 megawatts and becomes the largest single coordinated loss in the portfolio. The wind fleet, treated as one class, sheds 408. The nuclear group still leaves the lowest surviving inertia of any single class, at 3.57 seconds against 3.59 for the gas class, so it remains the inertia target while ceasing to be the magnitude target. Separation does not remove the adversary's choice. It moves it, and the owner of a separated campus should know where it moved. The last section of this part returns to that point.

Which dependencies can actually be separated

The partition says what separation is worth. It does not say what separation costs, or whether it can be bought at all. Taking the ledger's four dependency classes in turn gives a rough but honest answer for a modular campus. The digital class is the hardest of the four and has the section that follows to itself.

Physical dependencies can be separated at a price set mostly by land. Groups of modules can be placed at a standoff distance chosen against the threat tier, with independent heat removal where the site's water resource allows. Separation buys the assurance that one physical act at one location no longer reaches every group. It costs a larger footprint, longer internal connections, duplicated cooling works and, in some locations, a heat sink that cannot be divided at all. A campus that shares one ultimate heat sink shares it however far apart the reactor buildings stand, and that dependency is declared rather than engineered away.

Electrical dependencies can be separated within the campus at the cost of duplicated switchyards, transformers, and protection. At the boundary they can be separated at the cost of multiple points of interconnection, which is a question for the transmission owner as much as for the developer. Separation buys the assurance that no single switching station or transformer failure removes the

campus. The residual is on the grid side of the boundary. Three points of common coupling that feed one substation remain one class to an attack on that substation. That residual lies outside the developer's fence, and the end of this part takes it up.

Organizational dependencies are the cheapest to separate on paper and the most expensive to keep separated in practice. Staggered procurement across suppliers, crews rotated so that no two groups are worked to the same procedure at the same moment, and separate maintenance windows all give up efficiencies that a single-operator campus is otherwise built to capture. The least separable element of this class is one the design-basis method now names directly. DBA-MA-DiD §3.4 says that a class which fails together also recovers together, through one spare pool and one crew. Take a campus whose three groups draw long-lead transformers from one reserve and restoration labor from one contractor. It has three classes for the duration of the attack and one class for the duration of the recovery, and it is the recovery that sets the eighteen to twenty-four months. Separating the spare pool means holding spares against each group, which is capital held idle against an event that may never come. That is precisely the kind of cost a design basis exists to make visible.

Across these three classes the pattern is consistent. Separation is usually available, rarely free, and occasionally impossible. Where it is impossible, the dependency is declared and carried, and the method's residual limb does the rest.

Control independence without losing the common baseline

The digital class needs its own treatment for a reason that goes back to Part Three: firmware is the dependency with no physical extent. It has to be handled without contradicting the design-basis method, and the method imposes a constraint that System C, as Part Four described it, does not yet visibly meet.

DBA-MA-DiD §3.4 holds that adding redundant barriers does not restore adequacy under deliberate action, because an adversary who selects correlations simply re-selects to include the added one. Part Four described System C as having distinct control implementations across its three groups. Read quickly, that sounds like more control systems, and more control systems sounds like redundancy.

It is not what System C bought, and the reconciliation should be stated explicitly. The objection in DBA-MA-DiD §3.4 is to instances that share the dependencies that would defeat them. Adding such an instance changes nothing an adversary has to do. Distinct control implementations are not additional barriers. They are a change to the ledger at §3.2. The element recorded as shared across all twelve modules — one codebase, one firmware image, one update channel — is replaced by three elements, each shared only within its own group. That is the removal of a shared dependency. Its test is the one §3.4 already applies to any protecting measure: whether the action set that defeats one implementation also defeats the others. Where the implementations share a vendor, a codebase, an engineering toolchain, or an update path, it

does. The diversity is then nominal, and the closure returns one class. Where they share none of these, it does not, and the class count of three is earned. That is this paper's reading of the independence test, and the method has not adopted it. DBA-MA-DiD records the question as open at §7.7, sets out this reading, and says that a ruling is needed before a facility-class author first credits two such implementations. On the reading, the test already refuses diversity that is nominal and admits diversity that is real.

That reconciliation carries a cost an investor should see plainly. Three genuinely distinct control implementations mean three qualification efforts, three sets of procedures, three training programs, three spare inventories, and three bodies of operating experience, each thinner than one would have been. They also trade against verifiability, since a regulator or reviewer must now establish the adequacy of three implementations rather than one. Fleets standardize for sound reasons, and each of those reasons is a real cost of abandoning the standard.

Two alternatives preserve most of the common baseline, and either may be the better purchase.

The first keeps one firmware baseline across every group and protects one element independently of it. For a digital element, DBA-MA-DiD §3.4 names the forms of independent protection as segregation, determinism and formal verifiability. In a campus, this means a small, fixed-function element in each group's safety-significant path — the element that trips the group, isolates it or holds it in a safe state — segregated from the common baseline, deterministic in its behavior, and simple enough to be formally verified. The common baseline can then be defeated in all twelve modules at once without leaving any governing set without an independently protected element. The locally sovereign protection at the boundary is often the natural place for that element.

The second staggers updates, so that the three groups never run the same version of the common baseline at the same moment. This separates the update channel as a delivery mechanism. A malicious or defective update reaches one group, is observed there, and is stopped before it reaches the next. The approach is inexpensive and operationally familiar. Its limit should be stated with it. Staggering separates versions in time. It does not separate a latent flaw present in every version, and against an adversary who has already found such a flaw, the three groups remain one class. Staggering is therefore a partial separation of the digital class, useful against the update route and silent against a pre-positioned one, and the ledger should record it as exactly that.

Where any of these arrangements involves an authority that can command the island from outside — a supervisory system, an optimizing agent or a vendor's remote service — the governance of that authority is the subject of the Command Broker framework maintained in a companion series. This paper cites that framework and does not develop it.

The large load as a design input

The architecture exists to serve a large continuous load, and in serving it the island must not become the grid's problem. The July 2024 event supplies the reason. The facilities involved were not attacked and did not fail, yet they presented the grid with more than a thousand megawatts of simultaneous load loss because their protection responded alike.

The design basis for the hyperscale facility class already carries the relevant acceptance criterion. For the fourth postulated power-supply event, PE-4, a voltage excursion without loss of power, DBA-DC-FC1 §6.2 requires three things. The first is a documented inventory of all behind-the-meter protective equipment, with its settings and trip thresholds. The second is an assessment of the aggregate load that would drop out if that equipment operated at once. The third is confirmation that trip thresholds are not set inside the ANSI C84.1 Range A service band without engineering justification. For an islanded campus, this inventory is not paperwork for the load's operator. It is the input from which the grid-side contingency the campus represents is sized, and it belongs in the boundary's design basis alongside the generation.

The same concern has now reached the reliability organization in its strongest advisory form. On 4 May 2026, NERC issued a Level 3 Essential Action Alert on computational loads, with responses due on 3 August 2026. A Level 3 alert is the instrument NERC reserves for actions it considers essential to reliability. Whatever the final disposition of that alert, the developer of an islanded campus should expect to be asked the questions PE-4 already asks, and should expect the answers to be judged on the campus's aggregate behavior rather than on the settings of any single device.

Read through the partition, the point is a familiar one. Protective settings that respond alike are a shared dependency. They are organizational in origin and electrical in effect, and they group every facility using them into one class against a voltage disturbance that no adversary needs to cause. Diversity of settings, or coordinated ride-through, is the separation. The aggregate dropout assessment is the declaration.

The second tier, and why it may share the primary's class

The companion Fuel Cell position paper proposes a second tier beneath nuclear primary generation: onsite hydrogen fuel cells, fueled by hydrogen that the nuclear plant itself produces by electrolysis. Its Section 4.2 sets a requirement for that tier that is exactly right, namely no dependence on the same infrastructure that a grid-disrupting event would compromise. Its concluding section holds that the two tiers each address requirements the other cannot satisfy alone, and that, separately, each leaves a gap.

Against an adversary, the pairing that paper recommends does not meet its own requirement. Hydrogen produced by the plant shares the plant's site, its electrical path to the electrolyzers, its operator and, usually, its island controller. In the ledger, these are physical, electrical, organizational, and digital dependencies shared between the backup and the source it backs up,

and the closure places both tiers in one independence class. An action that defeats the primary at the site can defeat the second tier with it. An action that removes only the primary leaves the second tier drawing down a stored inventory that its only charging source can no longer replenish. Against persistence, the property Part One named among the four that break accident analysis, the second tier's endurance is whatever was banked before the attack.

None of this argues against the second tier. It argues that the second tier's claim to independence is a ledger claim like any other, and that the configuration the companion paper recommends, attractive as it is on carbon and on capital, is the configuration in which that claim is weakest. A second tier stored and controlled so that no element it shares with the primary can defeat it earns its independence. One that shares the primary's site, operator, and controller has the independence of a second room in the same building. This paper treats the question briefly, and the companion paper's next revision is the right place to develop it.

What the architecture does not solve

A separated campus behind split, locally sovereign and independently controlled boundaries is a materially better asset than any of Systems A, B or D. It is not a solved problem, and the investment case is stronger for saying what remains.

Grid-side concentration lies outside the fence. Three points of common coupling that terminate at one transmission substation, or on lines sharing one right-of-way, form one class against an attack on that substation or corridor. The developer can declare this and can seek diverse interconnection, but the remedy lies with the transmission owner and the interconnection process rather than with the campus design.

The adversary's choice moves rather than disappears. Once the nuclear campus is separated, as the arithmetic above shows, a gas lateral shared by combined-cycle generation becomes the portfolio's largest single coordinated loss, and it fails some twelve hours after it is cut rather than at the instant. Where a campus's own backup generation is gas-fired, a shared lateral is a dependency the campus inherits whether or not its generation plan mentions it.

The shared spare pool is the dependency least likely to be separated at any practical cost. A campus that separates everything else but draws its long-lead components from a common reserve has bought independence against the attack and given it up against the recovery.

Some dependencies cannot be separated at all and must be written down: a shared ultimate heat sink, a single qualified supplier of a component class, a single regional labor pool. Each is a declared residual under DBA-MA-DiD §3.5, and a developer whose list of residuals is empty has not finished the analysis. The output form that section requires is the right form for an investor as well. It is a statement that no coordinated loss within the stated threat collapses the campus without defeating an independently protected element, with every exception enumerated.

Part Seven takes up what adopting this architecture requires of a developer or investor, in sequence and at cost.

Part Seven: What Adoption Requires of a Developer or Investor

The sequence, and why order matters

Part Six described an architecture. This part describes what a developer must do to be entitled to claim it, and what an investor should ask to see before believing the claim. There are four obligations: declare the dependency set, have it reviewed, compute effective granularity from it, and dispose of every class that fails. They are taken in that order because each one depends on the one before.

The order is not bureaucratic tidiness. A count computed before the declaration has been reviewed is exactly the flattering number Part Five warned about: a figure with a stated derivation, resting on a set of dependencies nobody but its author has examined. A disposition decided before the count is known spends money against a problem that has not yet been located. Taken out of sequence, the four steps can each be completed and the analysis still be wrong.

Costs in this part are stated qualitatively, by what they are and against what they are paid, rather than in dollars. Figures would depend on the site, the design, the vendor, and the threat tier, and a number offered without those would mislead more than it informed. What can be said with confidence is where the money goes, whether it is paid once or keeps recurring, and how large it is relative to the asset.

Declare the dependency set

The first obligation is a written ledger. For every credited barrier and every generating unit, it records the substrate that unit or barrier depends on, sorted into the four dependency classes of DBA-MA-DiD §3.2 — physical, electrical, digital and organizational — and it marks each element as shared or not. For an islanded campus, the ledger must also record everything Part Six identified as crossing the boundary: control authority, spares, crews, heat sink where applicable, and communications, including vendor channels.

The cost is mostly engineering time, and against a nuclear campus it is small. The difficulty is not the time. It is getting the information. The dependencies that matter most in the digital and organizational classes are often held by vendors rather than by the developer: the lineage of a firmware image, the path an update travels, the subcontractors behind a control system, the single qualified supplier of a component. Vendors have commercial reasons not to disclose these, and a developer who asks late, after contracts are signed, has little leverage to insist.

The remedy is contractual, and it is cheap if applied early. A procurement specification can require disclosure of the dependencies the ledger needs as a condition of award. Written in at the start, the requirement costs a clause. Negotiated after award, it costs a change order, and some vendors will decline it altogether. Where a developer cannot obtain the information, the

conservative course is to declare the dependency as unknown and treat it as shared. That errs in the right direction, and it gives the vendor a reason to disclose.

Have someone other than the author review it

The second obligation is independent review of the declaration. This is the procedural remedy Part Five identified for the construct's central weakness: an undeclared dependency does not reduce the count; it disappears from view. No analytical check can find a dependency its author left out. Only a reader who did not write the ledger has a chance of noticing what it omits.

Several parties are positioned to review it. An owner's engineer can, though the independence is weaker when the owner commissions the review. A lender's technical adviser and an insurer's engineer answer to parties whose money rides on the result and who have no stake in a flattering number, which makes them better placed. In time a regulator may do it. The test for whether a reviewer is independent enough is the same test the ledger applies to everything else: the review counts if it is not defeated by the same pressures that shaped the declaration. A reviewer paid by the developer, reporting to the developer, and replaceable by the developer shares the developer's dependency.

The cost of review is small against the asset. The difficulty is finding reviewers with the right combination of knowledge. The declaration spans nuclear design basis, grid interconnection, control-system architecture, and supply chain, and few people are fluent in all four. In practice, review will be a small team rather than a single engineer, and the scarcest member will be the one who can read a firmware and update architecture with an adversary's eye.

Compute effective granularity and the adversarial vector

The third obligation is the simplest to perform and the easiest to overstate. Once the declaration has been reviewed, the transitive closure, the class count, and the adversarial resilience vector of Part Five follow by arithmetic. But the arithmetic is only as firm as its inputs, and Part Five has said that many of those inputs are estimates. The output for a real campus is therefore not one count but a range, and the design-basis method now requires that form at DBA-MA-DiD §3.2: the upper bound with every judged independence holding, the lower with every one failing, and the judgments that move the count between them named. A single figure is reported only where the review has shown that no judgment moves it. Until the method settles how the range is produced, its §7.6 asks that a declaration report both bounds, name every judgment that moves the count, and record who reviewed it. Parts Four and Six already show the form: System E holds three classes when its boundaries align and share no controller firmware, and one class when either condition fails. The independent review of the previous step exists largely to test the judgments that move the answer from one end of that range to the other.

What the computation produces is the page an investor should read beside the familiar resilience figures. The ordinary vector says what the portfolio is made of. The adversarial vector says what

its worst single coordinated loss costs: the largest output any one class can take away, the lowest inertia left after losing any one class, the longest class recovery time, and the number of classes. A developer who reports only the first has told an investor what was bought, not what it is worth against the event the architecture exists to survive.

Dispose of every class: protect one element or declare the residual

The fourth obligation is where the money goes. DBA-MA-DiD §3.4 allows two dispositions for a class that fails its adequacy test: protect at least one element of every governing set in a way the same attack does not defeat, or declare the consequence as a residual and carry it. Adding more units that share the dependency is not a disposition. Taking the four dependency classes in the order Part Six used gives a rough picture of what each disposition costs.

Physical separation is paid once, in land and in duplicated civil works. Standoff between groups of modules, separate cooling structures and independent heat removal all enlarge the site and lengthen its internal connections. Against the cost of the reactors themselves this is moderate. It is also the easiest separation to verify, since distance and walls can be measured. Where the site allows only one ultimate heat sink, no amount of money buys this separation, and the dependency goes straight to the residual list.

Electrical separation is paid once but depends on others. Separate switchyards and transformers within the fence are moderate costs the developer controls. Multiple points of interconnection with the grid are not wholly within the developer’s control. They depend on the transmission owner, the interconnection queue and the network’s configuration outside the fence, and they can add both cost and time to a project. As Part Six noted, separation at the fence is worth less if the separate connections converge on one substation beyond it.

Organizational separation is the one that keeps costing. Spares held against each group rather than in a common pool are capital that sits idle, year after year, against an event that may never come. Crews that do not all work to one procedure at one moment give up efficiencies a single-operator campus is built to capture. Staggered procurement gives up volume pricing. None of this is large in any single year. Over a twenty-year asset life it is the separation most likely to be quietly abandoned, because every budget review will find it easy to cut and hard to defend by anything that has happened.

Digital separation offers two routes at very different costs. Genuinely diverse control implementations, as Part Six described them, are expensive: multiple qualification efforts, procedure sets, training programs and spare inventories, and a heavier verification burden on whoever must establish their adequacy. The alternative is to keep one common baseline and protect a small, segregated, deterministic element in each group’s safety-significant path. That is far cheaper, because the element is simple by design, and it may well be the better purchase. Staggering updates costs little but, as Part Six noted, separates only the update route.

One observation cuts across all four classes. Separation is cheap on the drawing board and expensive after construction. Every item above costs a fraction as much when it is designed in as when it is retrofitted, and some items — site layout, heat sink, the number of interconnection points — cannot be retrofitted at all. The time to buy independence is before the design is frozen. A developer who defers the question to operations has usually answered it.

Where a class is not protected, the residual must be written down. Each residual states the dependency, the classes it joins, the consequence of their joint loss in the terms of the adversarial vector, and why the dependency was not separated. The last item matters most to an investor, because it distinguishes a residual accepted after costing from one never examined.

What can be asked voluntarily, and what would need a mandate

Not every obligation needs a regulator. The declaration, its independent review, the computed count and the residual list require no statute and no standard. They require only that someone with money at stake asks for them. An equity investor can make them a condition of commitment. A lender can write them into loan covenants and the scope of its technical adviser. An insurer can make them part of underwriting, and price the residual list. A power purchaser whose own operations depend on the campus can require them in the contract. Each of these parties already asks developers for engineering assurance. This adds a page to what they ask for.

Separation itself is harder to obtain voluntarily, and the reason is structural. Several of its costs fall on parties who do not receive its benefit. Separate spares protect the grid and the load as much as the owner. Diverse interconnection costs the transmission owner and the queue. Settings that ride through a disturbance rather than dropping out protect the grid at the facility’s expense, as the July 2024 event showed. Where cost and benefit sit with different parties, a market will under-buy the separation, and that is the ordinary case for a requirement. Whether any such requirement can be imposed, and by whom, is a question of legal authority. The companion policy paper takes it up. This paper does not answer it.

What an investor should ask for, and what an empty answer means

For an investor, all of this reduces to four documents: the declared dependency set, the independent review of it, the computed effective granularity and adversarial vector, and the list of declared residuals with their dispositions. None requires access to the site. All can be requested in diligence, and their absence is visible from outside.

The four also test each other. A declaration without a review is an assertion. A count without a declaration behind it is a number without a derivation. A single, unqualified count for a real campus is a warning of its own, since it suggests the estimates behind it were never varied. A residual list that is empty is the most informative answer of all, because no real campus has separated every dependency, so an empty list means the analysis stopped before disposition

rather than that disposition succeeded. The investor's question is not whether residuals exist. It is whether each one was costed and accepted, or never found.

Part Eight states what remains unresolved in this analysis, and who is in a position to resolve it.

Part Eight: What Remains Unresolved, and Who Can Resolve It

Why this part exists

A design basis states what it covers and what it does not. Part Seven asked a developer to produce a list of declared residuals and warned that an empty list means the analysis stopped early. This part applies the same standard to the paper itself. What follows is its own residual list: the questions the analysis reaches and cannot settle, each assigned to the party in a position to settle it.

Open in the method

The general form of the construct this paper relies on belongs to the design-basis method, DBA-MA-DiD. Four questions stood open there when this part was first drafted. One has since closed, and one has narrowed.

The first was that DBA-MA-DiD version 0.2 had not issued. It issued on 22 September 2026, and it carries the independence-class count at §3.2, the requirement that the dependency declaration be reviewed by someone other than its author, and the evaluation of consequence at the level of the class at §3.4. It remains an unpublished working draft. A reader should weigh those citations accordingly, and the method's owner is the only party who can change that.

The second is whether genuine diversity can count as independent protection. Part Six reconciled System C's distinct control implementations with DBA-MA-DiD §3.4 by applying that section's own test: a protecting measure counts if the attack that defeats one instance does not also defeat the other. That reading follows from the text, but the instrument has not adopted it. It now records the question as open at §7.7, sets out the reading Part Six applies, and states that the reading has not been adopted. A facility author will reach the question the first time a campus claims two control implementations that share no codebase, vendor, or update channel. The method's owner should rule on it.

The third is how a count of independence classes relates to a risk-informed analysis. The count is deliberately not a likelihood, and the method records at DBA-MA-DiD §7.3 that any link to frequency-based results must not bring attack probability back in by another route. The method's §7.3, as amended at version 0.2, now says that the count meets that caution as a quantification that is not a likelihood. It does not answer how that quantity should sit beside core-damage or large-release frequencies, and that question remains open with the method.

The fourth is how a count built on estimated dependencies should be reported, and here the method has moved half the distance. DBA-MA-DiD §3.2 now fixes the form, a bounded range with its deciding judgments named, which is the form Part Five recommends and Part Seven applies. It does not yet fix the convention for producing it: how many judgments to vary and whether jointly or one at a time, how to state confidence in each, and how a reviewer records that a judgment was tested rather than accepted. Those are open at §7.6. Until the method's owner settles them, developers who follow this paper will report ranges that agree in form and differ in construction, and comparisons between portfolios will be rougher than the construct deserves.

Open in the framework

Four limits belong to the companion papers in this family, and the author can resolve all four at their next revisions.

The parameter table assigns recovery time by source type, not by configuration. It therefore cannot express the finding of Parts Three and Six that a class sharing one spare pool and one crew recovers more slowly than the same machines held separately. A revision that lets recovery time vary with the declared dependency set would close that gap.

The table treats inverter-based inertia as zero. For today's grid, that errs in the safe direction. As grid-forming inverters become standard, it becomes simply inaccurate, and the framework will need to distinguish inertia guaranteed by physics from inertia that depends on a device behaving as specified, which is the distinction Part Three drew.

The Fuel Cell position paper's two-tier architecture needs its own dependency ledger. Part Six found that hydrogen produced by the primary plant can share that plant's independence class, which the paper's own requirement for the second tier does not allow.

The technical paper's forward-work section introduced the adversarial dual without naming the two parties. This paper now defines the term in Part One as a contest between defender and adversary reading the same parameter table, and the technical paper should carry the same definition.

Open in public authority

Five questions can be resolved only by public bodies. They are named here and not routed. Deciding which body can act on each, and under what authority, is the work of the companion policy paper.

The first is legal. Whether a minimum fuel independence, a reserve margin measured on delivered output, or a requirement to buy separation falls within what a reliability standard may require, or instead within the adequacy matters that Section 215 of the Federal Power Act withholds from the reliability organization and preserves for the states, needs confirmation by counsel. The companion papers no longer claim that no legislation is needed, and no such claim should be made until that confirmation is in hand.

The second is concentration on the grid side of the fence. Separate points of interconnection that converge on one substation or one corridor remain one class, and no developer can fix that alone. The remedy runs through transmission owners, regional planners and the interconnection process, which the Federal Energy Regulatory Commission has placed before the regional transmission organizations in its 2026 large-load proceedings.

The third is the behavior of large loads. NERC must file standards for computational loads, together with criteria for registering such loads, by 31 December 2026. Whether those standards reach the aggregate behavior of a campus, as Part Six argued they should, or only the settings of individual devices, will be decided in that process.

The fourth is the seam between nuclear licensing and grid reliability at a multi-module campus. A reactor trip is a protective function in the licensing basis, and a reliability standard cannot compel a unit to ride through a condition its licensing basis says it must trip on. When a condition presented to twelve modules at once warrants a trip at every one of them, each module behaves correctly, and the grid still loses the campus. Whether that is a nuclear safety question, a reliability question, or both, and which regulator governs the result, is unresolved. So is the status of controls belonging to a co-located load that can island or shed the campus while sitting under neither regime. The Nuclear Regulatory Commission and the Federal Energy Regulatory Commission are the parties who can settle it.

The fifth is the transformer reserve. The Government Accountability Office found in 2023 that the Department of Energy had studied the question twice without setting objectives, time frames or a plan. The recovery timeline that dominates every class in this paper's arithmetic will not shorten until a program exists, and that lies with the Department of Energy and with Congress.

Separately, where the island controller or any supervisory system can be commanded from outside the boundary, the governance of that authority belongs to the Command Broker framework, which a companion project maintains and this paper cites without developing.

Open with private parties

Three questions can be settled without any public body, by the parties who build, finance and supply the asset.

The first is cost. Part Seven describes the cost of separation qualitatively, by where the money goes and whether it recurs. Figures require site-specific engineering and vendor pricing that only developers and their suppliers hold. A developer who prices the dispositions of Part Seven for a real campus, and publishes even the ratios, would resolve more of the investment question than any further analysis in this paper could.

The second is reviewer capacity. The independent review Part Seven requires spans nuclear design basis, grid interconnection, control-system architecture and supply chain. The people who can do it end to end are few. Lenders' technical advisers, insurers' engineering teams and the

certifying bodies are the institutions best placed to build that capacity, and until they do, review will be the bottleneck.

The third is vendor disclosure. The declaration cannot be complete while firmware lineage, update paths and single-source suppliers are withheld. Procurement specifications can require disclosure. Whether vendors accept such a requirement as normal practice, or resist it until someone mandates it, will be settled in contracts, one project at a time.

What this paper claims, and what it does not

The paper claims four things. The properties that make a generating unit most valuable to the grid are the properties that make it the highest-consequence target, and this follows from the parameter table rather than from any assumption. The framework that recommends concentration has no term in which shared dependency appears, and a count of independence classes supplies one without introducing a likelihood. On that count, modular generation delivers resilience only when independence between groups is actually purchased, and the islanding boundary that makes the architecture attractive is itself the cheapest way to lose that independence. And the obligations that follow can be stated, sequenced and checked from outside, without access to any site.

The paper does not claim to know how likely any attack is, what separation costs on any real campus, or which public body has the authority to require it. It names no facility, and nothing in it describes one. Its conclusion about nuclear generation is the parent framework's conclusion, with a qualifier attached: nuclear passes every test the framework can express, and shared dependency is a test the framework could not express until now. Building to the recommendation without carrying that qualifier is how a portfolio arrives at maximum measured resilience and maximum unmeasured exposure at the same moment.

The policy and infrastructure-protection treatment announced on this paper's cover takes these findings to the people who decide what the country requires of its infrastructure.

Revision History

Version	Date	Description
0.15	17 October 2026	Citation form. In Part Six, under "The islanding boundary," the pointer to the Predictive UFLS Ride-Through position paper no longer names that paper's version, under the family rule that companion papers are cited by title; the index in the Always-On Paper Family folder records current versions. The erratum to versions 0.12 and 0.13 keeps its version references, since it records what particular versions said. No change to the argument.
0.14	3 October 2026	Conformed to the October 2026 Edition of the Foundational Definitions. In

Version	Date	Description
		Part Six, under “The islanding boundary,” the passage on local sovereignty now states FD-LS Section 3.2 as issued: the physical-law condition as a fixed function of measurements and setpoints with no learned element, and the third condition in its two limbs, with the governed act. It draws the design consequence itself instead of pointing to the Predictive UFLS Ride-Through paper’s Section 4.6 for it, adds that the boundary protection’s settings change only through a governed act, and adds that a learned model is not locally sovereign however locally it runs. It cites that paper’s Section 4.6 only as corrected at version 0.8. The FD-LS Section 4 paragraph that follows is correct and is unchanged. An erratum before A Note on Scope names the version 0.12 and 0.13 deposits and quotes the pointer in error and the superseded statement of the conditions. Publication handling under the CR-PB draft: versions 0.12 and 0.13 are DOI-bearing and stay frozen; this version is the successor and goes forward as a new Zenodo version under the concept DOI, and the cover carries the concept DOI until this version’s DOI is reserved. No change to the argument or the arithmetic.
0.13	28 September 2026	Conformed to DBA-MA-DiD v0.2, issued on 22 September 2026 as an unpublished working draft, on Project 6’s return note of that date. Part One records the issue. Part Five records the closure, the count and the declarant review as carried at §3.2, and the bounded-range form as adopted there. Part Six cites the recovery sentence to §3.4 as issued and records that §7.7 holds the diversity question open with this paper’s reading stated and not adopted. Part Seven aligns the computation step with the bounds of §3.2 and the interim practice of §7.6. In Part Eight the first open item in the method is closed, the third cites §7.3 as amended, and the fourth narrows to the convention open at §7.6. Publication handling, stated under the CR-PB draft: the master is corrected and republished as a new Zenodo version under the concept DOI; v0.12 stands frozen under its own version DOI, 10.5281/zenodo.22818332, and carries no erratum, because it was accurate on its date. Section headings now carry the Heading 1 and Heading 2 styles, redefined to the house specification, so the document has a navigable outline and PDF bookmarks; their appearance is unchanged. A copyedit pass adds serial commas and commas after introductory phrases throughout. The inertia constant is given with its unit, seconds, where first stated. A Note on Scope now tells the reader that the paper numbers none of its own sections and which document each section number belongs to, and the first section reference in each paragraph names its document. No change to the argument or the arithmetic.
0.12	17 September 2026	Publication pass for the Zenodo deposit of September 2026. The cover carries the citation block with the reserved version DOI, describes the document as a complete draft rather than a draft for author review, and names the policy treatment, which now exists, instead of describing it as planned. A scope review was run against the cleared line before minting: the document names no utility, facility, site, interconnection, vendor or product, and states no configuration, vulnerability or method particular to a real installation. No change to the argument.

Version	Date	Description
0.11	17 September 2026	Alignment run across the always-on paper family. Part Six adds the 22 July 2026 load transfer event of about 3,869 megawatts beside the July 2024 event. Under the author's 17 September 2026 direction, companion papers are cited by title only and the family index carries versions; this document already cited its companions by title, and its references to DBA-MA-DiD keep their versions because the distinction between v0.1 and the unissued v0.2 carries meaning. Part Eight no longer attributes to the companion papers a claim that no legislation is needed, since those papers have been corrected.
0.10	16 September 2026	The dependency declaration is now described as built largely from estimates, at the author's direction. Part Five adds a paragraph distinguishing near-factual entries from judgments and recommends reporting effective granularity as a bounded range with its deciding judgments named. Part Seven's computation step is rewritten to match, and its investor test adds that a single unqualified count is itself a warning. Part Eight adds a fourth open item in the method: no settled convention yet for reporting a count built on estimates.
0.9	16 September 2026	Revision history moved from after the cover to the end of the document, at the author's direction. No change to the text.
0.8	16 September 2026	Part Eight drafted on the structure the author approved: open items in the method (DBA-MA-DiD v0.2 unissued, diversity under §3.4, the quantification boundary), in the framework (recovery time by configuration, synthetic inertia, the second-tier ledger, the dual definition in the technical paper), in public authority (Section 215, grid-side concentration, computational-load standards, the NRC and FERC seam at a multi-module campus, the transformer reserve, with the Command Broker cited only), and with private parties (cost figures, reviewer capacity, vendor disclosure), closing with what the paper claims and does not. All eight parts are now drafted.
0.7	16 September 2026	Part One now defines the dual as a contest between the defender and the adversary reading the same parameter table, and states that the term is used in that plain sense rather than in the formal sense of optimization theory. Added at the author's direction.
0.6	15 September 2026	Part Seven drafted on the structure the author approved: the four obligations in sequence (declare, review, compute, dispose), the qualitative cost of each disposition by dependency class, the design-in versus retrofit asymmetry, what investors, lenders, insurers and purchasers can require without a mandate and why separation itself may need one, and the four documents an investor should request. Costs are stated qualitatively at the author's direction. Part Eight remains outlined.
0.5	15 September 2026	Part Six drafted: the islanding boundary, Systems D and E worked and recomputed programmatically against Table 1 and Part Four, the separability of the four dependency classes, the reconciliation of System C's distinct control implementations with DBA-MA-DiD §3.4, the large

Version	Date	Description
		load as a design input, a brief treatment of the second tier, and what the architecture leaves unsolved. The author selected the worked Systems D and E, a brief second-tier treatment, and citation-only treatment of the Command Broker. Part One's note on the standing of DBA-MA-DiD now records that v0.2 is determined and not yet issued. Parts Seven and Eight remain outlined and unwritten.
0.5	15 September 2026	Dependents reading, owed as condition three of the transfer accepted by Project 6 on 14 September 2026. Seventy-six live Word and markdown masters in Project 8's tree, excluding this document, the archive, correspondence and session folders, were extracted in full and searched for independence class, independence count, effective granularity, G_eff, R_adv, transitive closure, dependency set, the class-level evaluation in its formulations, and this document by title. One dependent family was found: Keeping the Lights On, Annotated Outline v0.3 to v0.5, which takes effective granularity from this document by title for the generation-sector reading. Its general form is to be cited to DBA-MA-DiD §3.2 when that paper is drafted. No other document carries the constructs. Word matches elsewhere on islanding boundary, shared dependencies and class-level were read and are unrelated uses.
0.4	14 September 2026	Parts One through Five. Earlier versions carried no revision table; this table begins at v0.5.